



Approfondimento sui Linguaggi di Programmazione

Massimo Esposito



Premessa

Premessa

- La lezione esplora le principali caratteristiche dei **linguaggi di programmazione**, analizzandone la struttura e le componenti fondamentali.
- Vengono approfonditi concetti come il **lessico**, la **sintassi**, la **semantica** e la classificazione dei linguaggi secondo diversi **paradigmi**.
- Inoltre, si esaminano in dettaglio i processi **di traduzione** dei linguaggi e sono state evidenziate le principali tipologie di **errore** che emergono durante la scrittura e l'esecuzione di programmi.

Sommario

1

Caratteristiche dei
linguaggi di
programmazione

2

Paradigmi di
Programmazione

3

Traduzione dei
Programmi

4

Conclusioni e sintesi



Caratteristiche dei linguaggi di programmazione

Definizione di linguaggio di programmazione

- I linguaggi di programmazione sono **sistemi formali** utilizzati per scrivere istruzioni comprensibili da un computer.
- Essi si basano su una combinazione di **lessico**, **sintassi** e **semantica** per definire comportamenti e strutture del codice.
- L'obiettivo principale è **tradurre** l'intento umano in comandi eseguibili da una macchina.

Lessico

Il **lessico** di un linguaggio di programmazione definisce le regole per la formazione di parole valide nel linguaggio.

I componenti principali del lessico sono:

- **Parole chiave** (es. if, else, while, ...)
- **Identificatori** (es. nomi di variabili,...)
- **Operatori** (es. +, -, *, /, &&, ||, =, ...)
- **Delimitatori** (es. parentesi tonde, parentesi graffe, punti e virgola, ...)
- **Letterali** (es. numeri, stringhe, caratteri, true, false, ...)
- **Commenti** (es. //, #, /*, */ , ...)

Sintassi e grammatica

- La **sintassi** definisce come le parole devono essere combinate per formare **frasi** valide nel linguaggio.
- Essa include regole simili a una **grammatica formale**, usata per costruire istruzioni comprensibili.
- Una sintassi ben definita è cruciale per evitare **ambiguità** nel codice.

Semantica

- La **semantica** di un linguaggio descrive il **significato** delle istruzioni, ovvero cosa fa un programma quando viene eseguito.
- A differenza della sintassi, la semantica si concentra sul **comportamento** e sul **risultato** delle istruzioni.
- Comprendere la semantica è essenziale per scrivere codice che funzioni come previsto.

Tipologia di Linguaggi di programmazione

Linguaggi di basso livello

- linguaggi più difficili da leggere e scrivere per l'essere umano, in quanto sono specifici per l'hardware

Linguaggi di alto livello

- linguaggi generalmente indipendenti dall'hardware
- sono linguaggi tipicamente più facili da leggere e scrivere in quanto più vicini al linguaggio umano

Linguaggi di basso livello

Caratteristiche

- Si pongono al livello della macchina comunicando direttamente con il processore.
- Utilizzano codici operativi (binari o mnemonici) specifici per il processore per cui il linguaggio viene progettato.
- Comportano per il programmatore difficoltà di scrittura e di verifica del corretto funzionamento del codice.

Esempi

- Linguaggio macchina, linguaggio Assembly.

Linguaggio macchina

Linguaggio più semplice da comprendere per la CPU

- stretto legame con l'hardware
- alta velocità di esecuzione ed alto grado di ottimizzazione nell'uso delle risorse hardware

Non trasportabilità del codice tra processori differenti

- un programma scritto per una CPU non è eseguibile su una CPU diversa con caratteristiche diverse

Linguaggio macchina

Difficoltà di programmazione

- gran numero di comandi per singole istruzioni
- istruzioni espresse sotto forma di sequenze di bit

Linguaggi assemblativi

Sostituiscono alle sequenze di bit dei codici mnemonici più facili da interpretare e ricordare per l'essere umano

- mantengono uno stretto legame con il linguaggio macchina
- tipicamente, un assemblatore si occupa di tradurre i codici mnemonici del linguaggio assemblativo nelle sequenze di bit del linguaggio macchina

Esempi

- linguaggio Assembly

Linguaggi di alto livello

Caratteristiche

- fanno uso di istruzioni più sintetiche e vicine al linguaggio umano per esprimere i procedimenti di calcolo
 - pseudo-linguaggio umano basato su parole chiave o codici operativi ispirati quasi esclusivamente alla lingua inglese
 - rendono l'attività di programmazione più semplice
- necessitano di un "interprete" (detto anche traduttore) per la traduzione in istruzioni di basso livello interpretabili dalla CPU

Esempi

- C/C++, Python, Java

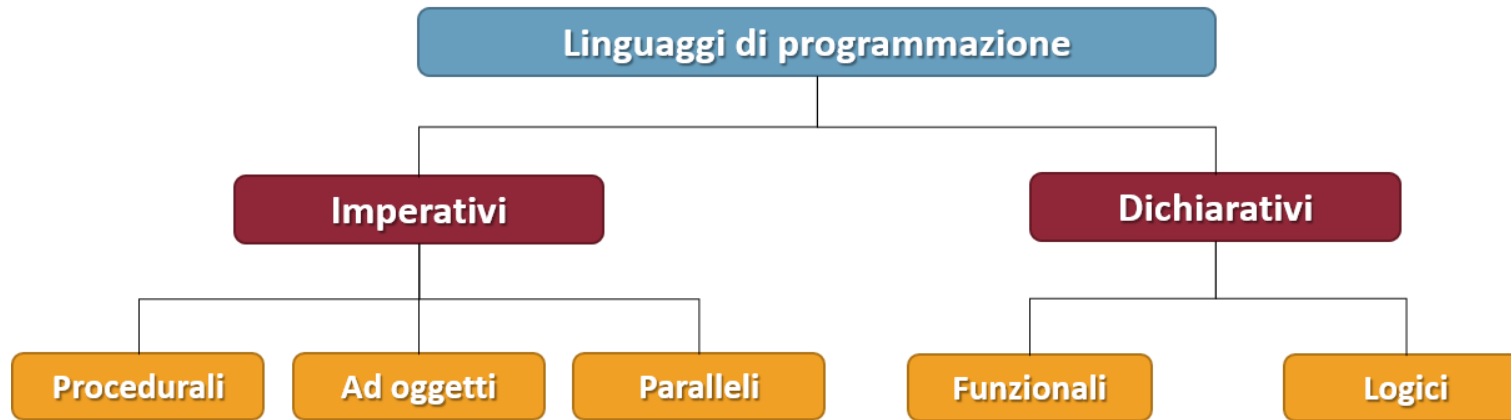


Paradigmi di Programmazione

Cosa sono i Paradigmi di Programmazione

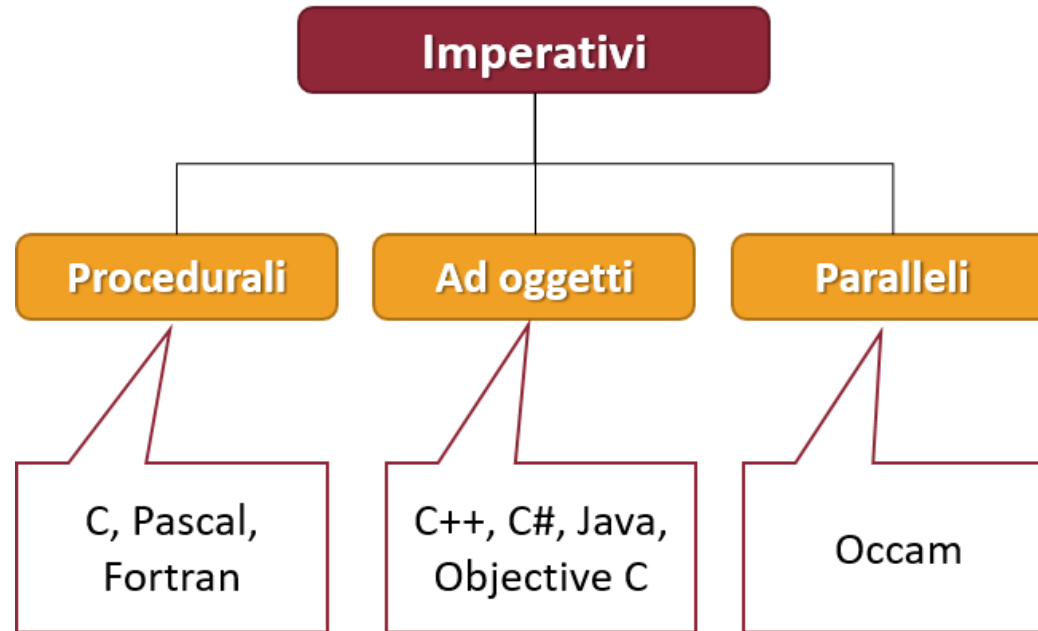
- Un **paradigma di programmazione** è un modello o un insieme di principi che definisce lo stile e l'approccio alla scrittura di programmi, fornendo concetti e strutture per risolvere problemi specifici.
- I paradigmi di programmazione influenzano **lo stile** e **l'approccio** con cui i programmatori pensano e organizzano il codice.
- Comprendere i paradigmi aiuta a **scegliere lo stile** più adatto al problema.

Principali Paradigmi di Programmazione



- La scelta del paradigma di programmazione dipende dal **problema** che si vuole risolvere e dalle **preferenze** del programmatore.
- Alcuni linguaggi di programmazione possono supportare **più paradigmi**, permettendo ai programmatori di utilizzare gli strumenti più adatti.

Linguaggi imperativi



- Il problema viene risolto mediante **una sequenza ordinata di istruzioni** che cambiano lo stato del programma attraverso assegnazioni di variabili e strutture di controllo
- L'enfasi è sui dati (variabili, strutture dati, oggetti) e sui modi per processarli (tramite procedure, funzioni, metodi)
- **Esempi: C, C++, Java**

Programmazione Procedurale

Si concentra sulla **modularità del codice**, decomponendo i compiti da svolgere in blocchi di codice detti **sottoprogrammi** o **moduli**.

- I programmi sono tipicamente strutturati in termini di procedure e funzioni.

Esempi: C, Pascal

Programmazione Orientata agli Oggetti (OOP)

Si concentra su concetti quali **l'incapsulamento**, **l'ereditarietà**, il **polimorfismo**, e **l'astrazione del codice**.

- modella un problema come un insieme di **oggetti**, che incorporano dati e comportamenti, che si scambiano messaggi
- gli oggetti sono istanze (concrete) di tipi di dato astratti (le classi) definiti dal programmatore

Esempi: Java, C++, Python

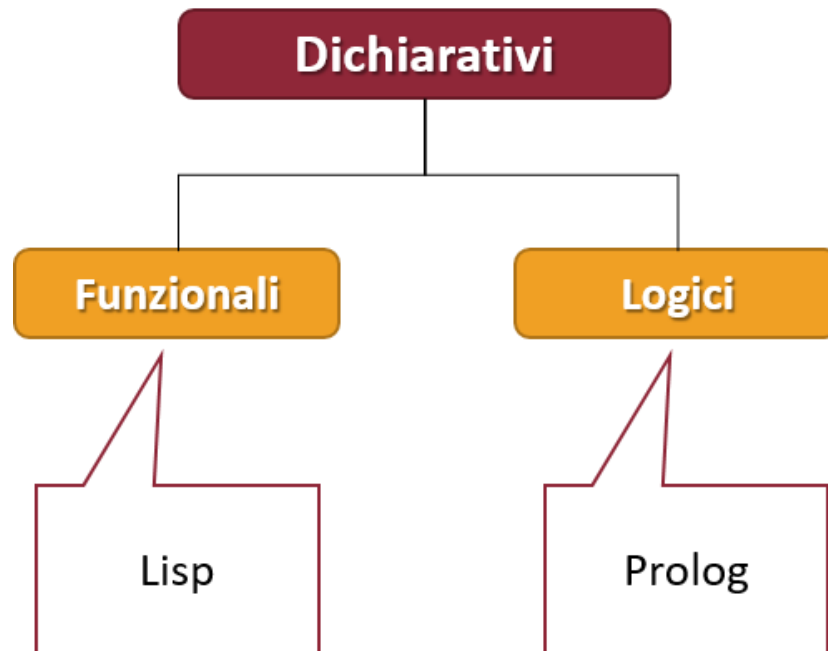
Programmazione Parallela

Si concentra sull'**elaborazione dei dati in parallelo** anziché con il classico approccio sequenziale

- un problema viene decomposto in parti che posso essere eseguite in maniera **concorrente** (in parallelo) su CPU differenti
- permette di descrivere formalmente algoritmi non sequenziali

Esempi: Occam, C/C++, Python

Linguaggi dichiarativi



- **Paradigma orientato al problema**, cioè a "cosa" il programma deve fare per risolvere un dato problema, piuttosto che sul "come" farlo.
- **Non si specifica la sequenza esatta delle operazioni**, come invece avviene nella programmazione imperativa.

Programmazione Funzionale

Propone un modello di esecuzione basato su **funzioni matematiche pure**.

- Si ispira direttamente alla logica matematica e alla teoria delle funzioni, portando a soluzioni eleganti e dichiarative per problemi complessi.
- L'elaborazione si articola come una serie di valutazioni di funzioni, evitando l'uso di variabili mutabili e stati globali.

Esempi: Haskell, Lisp

Programmazione Logica

Si basa sulla **logica formale** per descrivere il comportamento dei programmi in termini di **fatti** e **regole**.

- I programmi sono composti da un insieme di dichiarazioni che rappresentano **conoscenze di base** (fatti) e **inferenze condizionali** (regole), utilizzate per dedurre nuove informazioni.
- L'esecuzione avviene mediante un meccanismo di **inferenza** che cerca di rispondere a una domanda (query) combinando le informazioni disponibili.

Esempi: Prolog



Traduzione dei programmi

Processo di traduzione

- La **traduzione del codice** consiste nel trasformare un linguaggio ad alto livello in un linguaggio comprensibile dalla macchina.
- Questo processo può avvenire tramite un **compilatore** o un **interprete**.
- La traduzione è cruciale per permettere l'esecuzione su diverse **architetture hardware**.

Compilatori

- Un compilatore è un programma che traduce tutto il codice sorgente in linguaggio macchina **prima dell'esecuzione**.
- I compilatori producono eseguibili **efficienti** e **indipendenti dal codice sorgente**.
- **Esempi: Fortran, C, C++**

Interpreti

- Un interprete **esegue direttamente** il codice sorgente istruzione per istruzione.
- Questo approccio offre **maggiore flessibilità e portabilità** ma può risultare più lento rispetto alla compilazione.
- **Esempi: Python, PHP, Java**

Errori nei programmi

Errori che non portano all'esecuzione del programma

- Errori nella compilazione del codice sorgente
 - errori di sintassi nella scrittura delle istruzioni
- Errori nel collegamento di oggetti e librerie
 - uso di un elemento in un sorgente diverso dal modo in cui è definito in un altro sorgente
- Errori nel caricamento dell'eseguibile in memoria
 - un programma necessita di più memoria di quella esistente

Errori nei programmi

Errori che portano alla terminazione improvvisa del programma se non gestiti

- Errori nell'esecuzione del programma
 - es.: divisione per 0, operazioni ripetute all'infinito (loop)



Conclusioni e sintesi

Conclusioni e sintesi

- La lezione ha illustrato le basi dei **linguaggi di programmazione**, approfondendo lessico, sintassi e semantica.
- È stata evidenziata la distinzione tra **linguaggi di basso e alto livello**. Sono stati presentati diversi **paradigmi**, ciascuno con un proprio modello risolutivo, con **imperativo** e **dichiarativo** che ne rappresentano le principali forme.
- Infine, si è discusso del processo di **traduzione** attraverso compilatori e interpreti ed è stato analizzato il ruolo degli **errori** nello sviluppo e nell'esecuzione di programmi.